Business Process Prediction, Simulation and Optimization

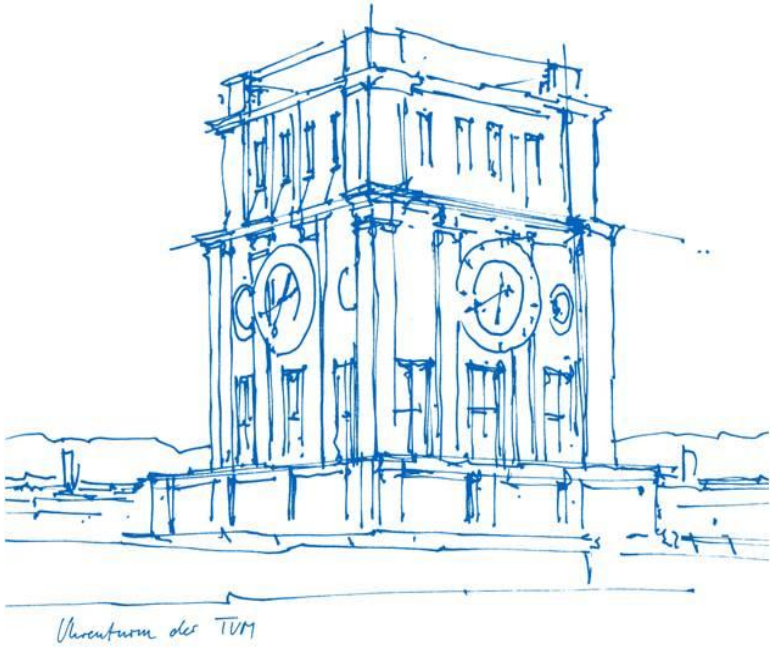
3. Process Mining II

Henryk Mustroph, Prof. Dr. Stefanie Rinderle-Ma

Chair of Information Systems and Business Process Management (I17), TUM School of CIT

Technical University of Munich

Garching, 29 October 2026



Overview

- You **know how a process can be modeled** using different modeling techniques. The chosen **model type depend on the expected output and the purpose of the model**.
- You know the **intent of process discovery and several process discovery algorithms**. Different discovery algorithms have different advantages and return different process model types.

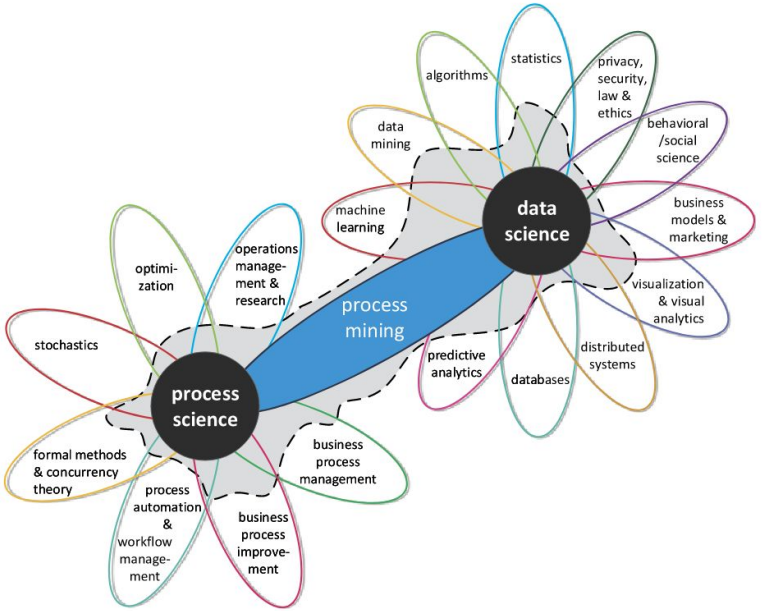
> **Questions**: How to evaluate a process model goodness? and how to know why and when certain branches in the process model are taken during simulation?

Process Mining

What Is Process Mining?

Definition from van der Aalst's textbook

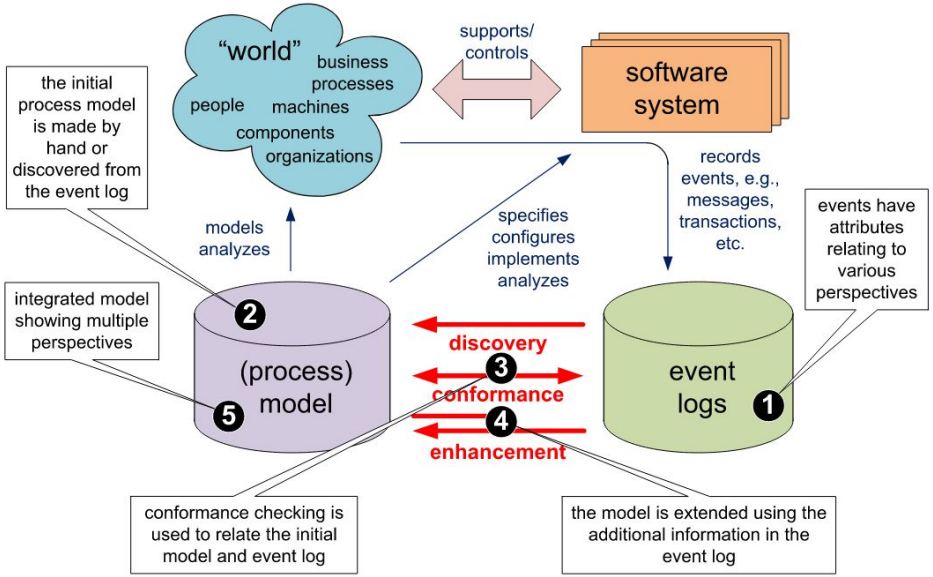
> “The goal of process mining is to turn event data into insights and actions. Process mining is an integral part of data science, fueled by the availability of data and the desire to improve processes.” [1, p. 2]



Source: [1]

Main Types of Process Mining

Discovery, conformance and enhancement



Source: [1]

Three Main Types

Each type relates an event log to a process model

1 Discovery

Produces a model based on an event log without using any a-priori information.

2 Conformance

An existing process model is compared with an event log of the same process.

3 Enhancement

Improve an existing process model using information about the actual process recorded in an event log.

Conformance Checking

Conformance Checking

Two viewpoints on mismatches between log and model

Conformance checking aims to assess how much of the behavior (control flow only) observed in the event log is reflected in the process model.

Wrong Model

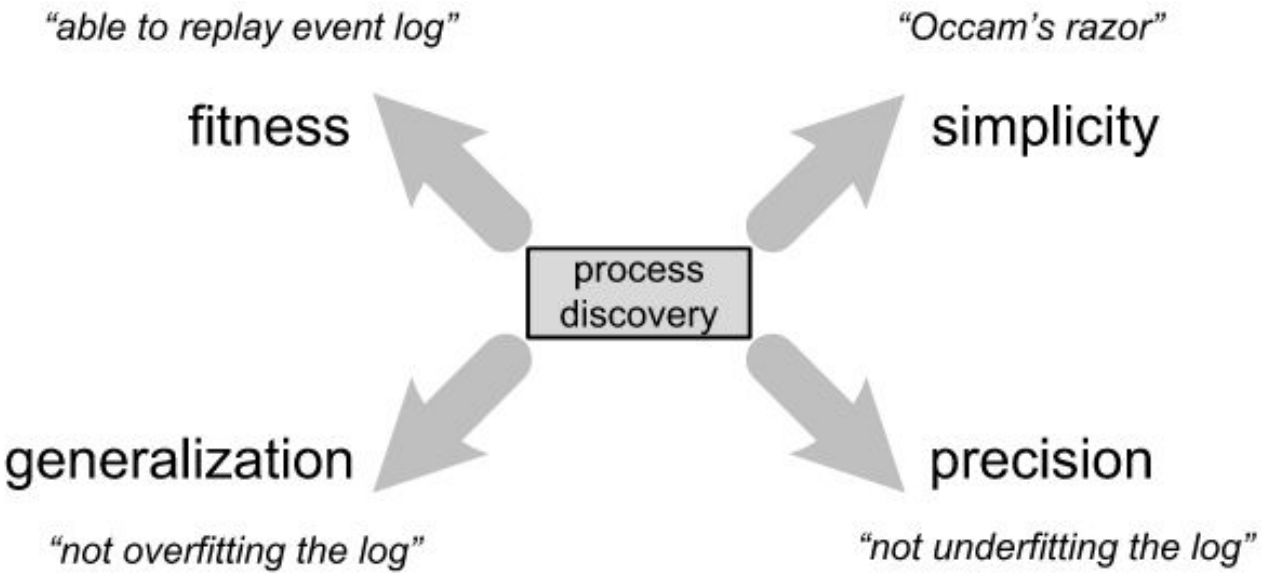
The model is wrong and does not reflect reality.

Deviating Cases

Cases/ traces deviate from the model (outliers).

Quality Criteria

What is a good process model?



Source: [1]

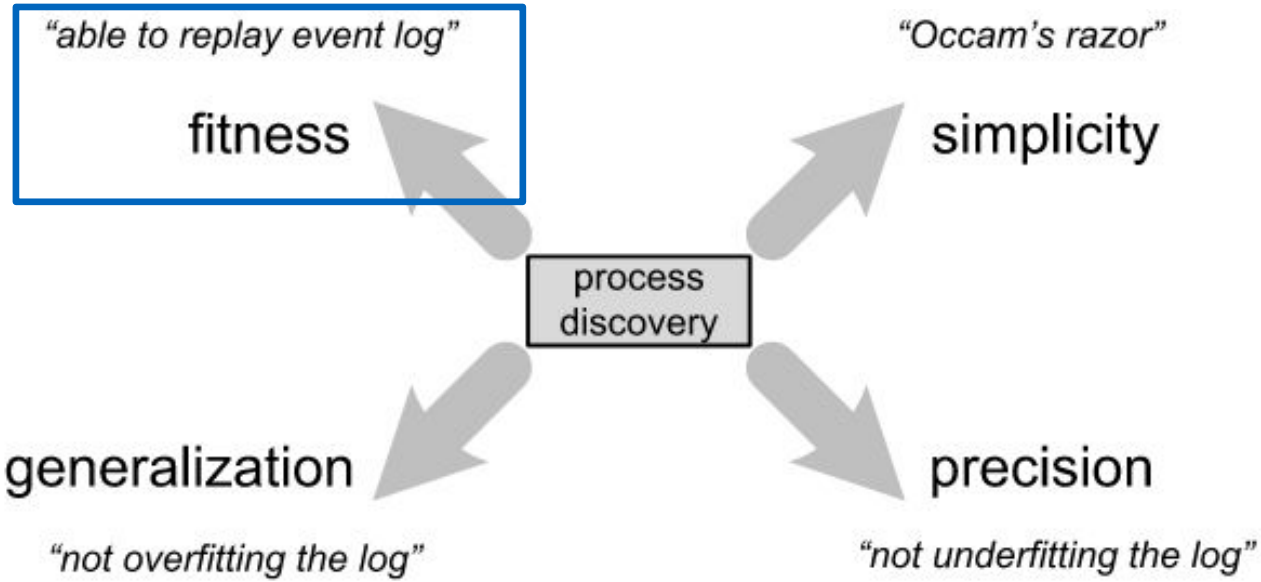
Quality Criteria

Four criteria to judge a process model

Fitness	How much of the observed behavior can be reproduced by the model (result from conformance checking algorithms).
Precision	How much behavior does the model allow that was never observed.
Generalization	How well is the model expected to handle unseen but legitimate future behavior.
Simplicity	How understandable is the model to a human.

Quality Criteria

Next up: fitness



Source: [1]

Fitness

How to measure fitness?

Token-Based Replay

Idea: take each trace from the event log, “replay” it on the model by firing the corresponding transitions, and bookkeep what goes wrong.

Alignment-Based Approach

Idea: Instead of replaying the, an alignment treats the problem as a global optimization: find the cheapest way to match the observed trace against some execution of the model.

Source: [5], [6]

Token-Based Replay Approach

Replay a trace on the Petri net and store four token counts

p (produced)	Tokens that the replay produces in places (including the initial token in the source place).
c (consumed)	Tokens that the replay consumes from places (including the final token in the sink place).
m (missing)	Tokens that should have been in a place to enable the next transition but were not, so the replay artificially injects them to keep going. Indicate the trace tried to do something the model did not enable.
r (remaining)	Tokens left in the net after replay terminates that were never consumed. Remaining tokens indicate the trace failed to do something the model required.

Source: [5]

Fitness: Token-Based Replay Approach

How to measure fitness

Metric 3 (Fitness) *Assume a given event log E and a labeled Petri net PN (cf. Chapter 2). For each event sequence $\sigma \in S = \alpha(E)$, let m_σ be the number of missing tokens, r_σ the number of remaining tokens, c_σ the number of consumed tokens, and p_σ the number of produced tokens during log replay of that event sequence in the given Petri net. The token-based fitness metric f is defined as follows:*

$$f = \frac{1}{2}\left(1 - \frac{\sum_{\sigma \in S} S(\sigma) \cdot m_\sigma}{\sum_{\sigma \in S} S(\sigma) \cdot c_\sigma}\right) + \frac{1}{2}\left(1 - \frac{\sum_{\sigma \in S} S(\sigma) \cdot r_\sigma}{\sum_{\sigma \in S} S(\sigma) \cdot p_\sigma}\right)$$

A perfect replay yields m = r = 0 and fitness = 1.

Source: [3], [5]

Token-Based Replay Approach

Limitations of token-based replay

Non-determinism	Consider a model with an exclusive choice with two branches. Both start with A and continues with B, the other continues with C. Consider the current trace having <A,C> as events. What would happen with token replay?
Unreliable results	Token-based replay does not deliver reliable and deterministic results if the trace deviates from the process model. A single deviation could lead to a chain of artificial follow on deviations.
Token flooding	In Petri nets with many loops or/ and silent transitions.

> These limitations are solved with the alignment-based approach!

Alignment-Based Approach

Find the cheapest way to match the observed trace against the model

Moves on the Synchronous Product

- **Synchronous moves (a,a):** Activity can be replayed in both the trace and the model.
- **Log moves (>>,a):** Activity is only existent in the log but in the process model.
- **Model moves (a,>>):** Activity is only existent in the model but not in the log.

Optimization Problem

- Find the firing sequence in the synchronous product that incurs the minimal cost for reaching the final marking
- Each transition comes with a cost for firing, e.g.: 1 : all log/ model moves, 0 : sync moves and for silent transition moves.

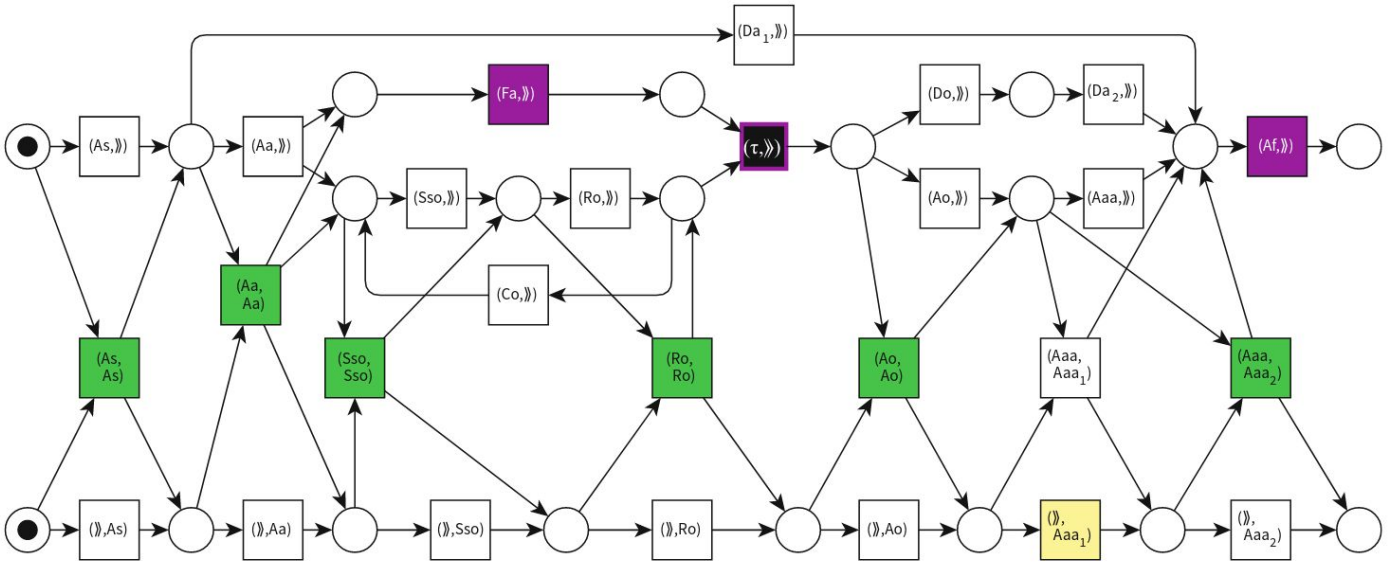
Example: Synchronous Product

Synchronous product of a process model and a trace

Model

Synchron

Trace



Fitness: Alignment-Based Approach

How to measure fitness

A model with good *fitness* allows for most of the behavior seen in the event log. A model has a perfect fitness if all traces in the log can be replayed by the model from beginning to end. To quantify fitness, we can directly apply the distance function δ on the optimal alignments provided by the "oracle" λ_M. The total alignment cost for event log L and model M is

$$fcost(L, M) = \sum_{\sigma_L \in L} \delta(\lambda_M(\sigma_L))$$

Source: [6]

Fitness: Alignment-Based Approach

How to measure fitness

Often, we would like to express fitness as a number between 0 (very poor fitness) and 1 (perfect fitness). There are several ways to normalize the alignment costs. One approach is to divide $fcost(L, M)$ by the maximal possible cost. Let us assume that for $x \in A_L$ and $y \in A_M$: $\delta(x, y) = \infty$ if $x \neq y$, i.e., we only allow a move on both model and log (x, y) if $x = y$. In this case, the worst-case scenario is that there are just moves in the log and moves in the model (and never both) in the optimal alignment. $move_L(L) = \sum_{\sigma_L \in L} \sum_{x \in \sigma_L} \delta(x, \perp)$ is the total cost of moving through the whole log without ever moving together with the model. $move_M(M) = \min_{\sigma_M \in \beta(M)} \sum_{y \in \sigma_M} \delta(\perp, y)$ is the total cost of making moves on model only. Note that we consider the "least expensive path" because an optimal alignment will try to minimize costs. This results in the following definition of fitness:

$$fitness(L, M) = 1 - \frac{fcost(L, M)}{move_L(L) + |L| \times move_M(M)}$$

Source: [6]

Alignment-Based Fitness

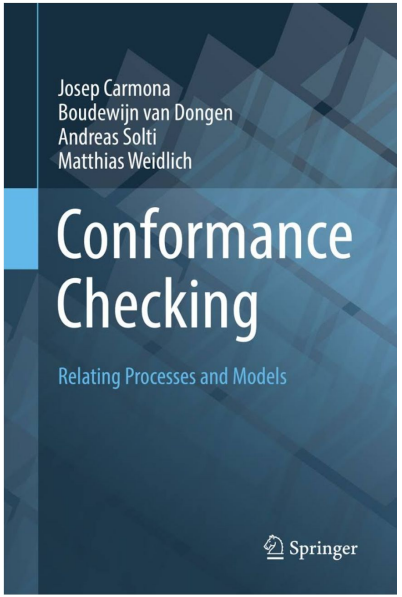
Advantages of the alignment-based approach

Global optimality	An alignment is provably the minimum-cost explanation of the trace given the model
Interpretable diagnostics	The output is not a count of stranded tokens but a precise sequence of moves
Robustness on hard model constructs	Concurrency, loops, duplicate labels, silent transitions, and non-free-choice patterns all break or mislead token replay. Alignments handle them correctly because the search reasons about full markings, not about local enabledness.

Recommendation

A book on conformance checking

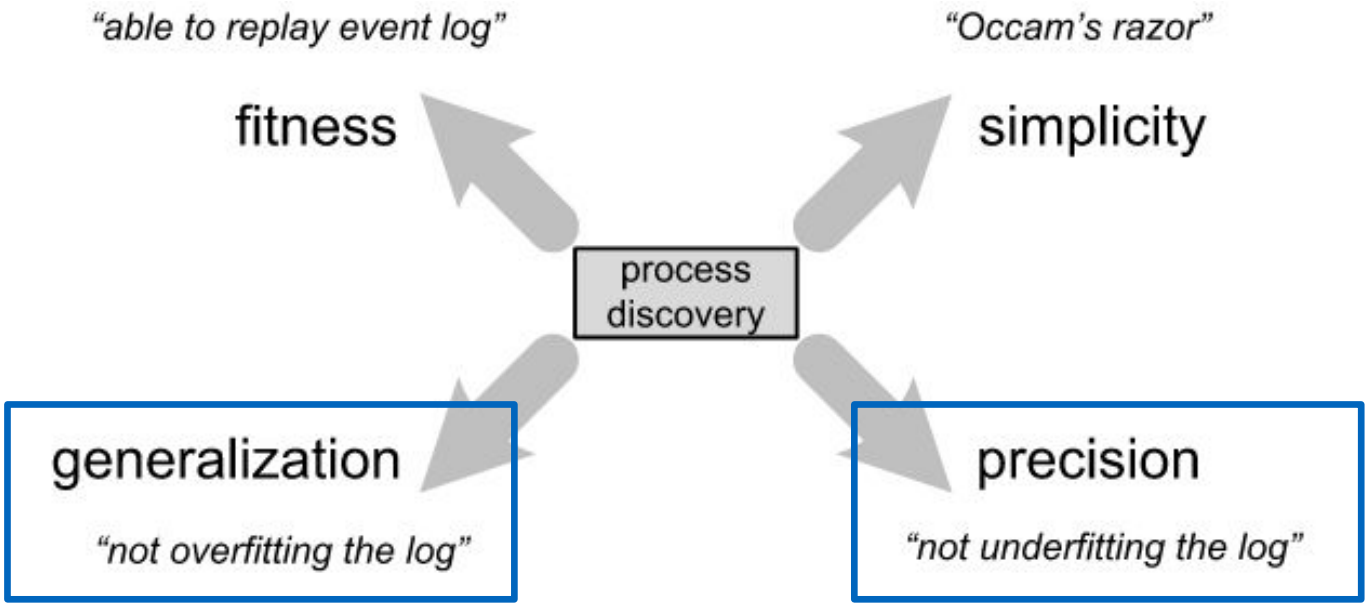
> **Well written book**
> Introduces basics and all presented conformance checking algorithms



Source: [4]

Quality Criteria

Next up: precision and generalization



Source: [1]

Precision and Generalization

Underfitting versus overfitting

Precision: Not Underfitting

- A precise model is tight around the log.
- An imprecise model is underfitting: it's too permissive, allowing arbitrary interleavings or paths that have no support in the data.

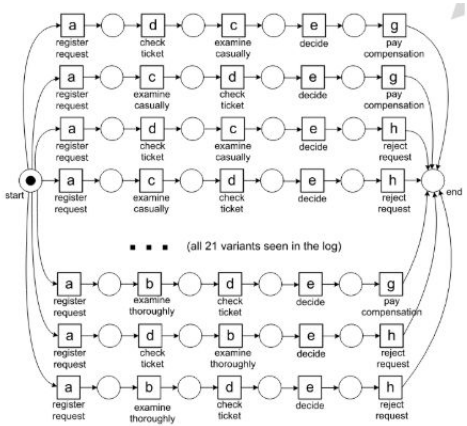
Generalization: Not Overfitting

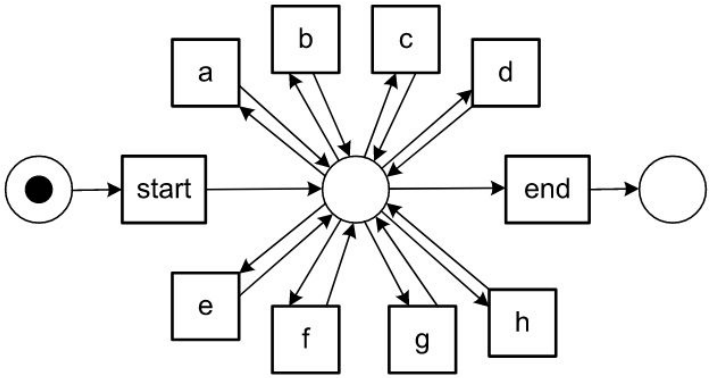
- A well generalized process model handle unseen but legitimate future behavior.
- A model generally that has perfect fitness and precision allows no unobserved behavior and therefore generalizes terrible.

Precision and Generalization

Two example process models

Which process model has high precision? Which process model has high generalization?





Source: [1]

Precision and Generalization

How to measure precision / generalization?

Metric 4 (Simple Behavioral Appropriateness) *Given an event log E and a labeled Petri net $PN = (P, T, F, L, l)$, for each event sequence $\sigma \in S = \alpha(E)$, let x_σ be the mean number of enabled transitions during log replay of that event sequence in the given Petri net[3]. The simple behavioral appropriateness metric a_B is defined as follows:*

$$a_B = \frac{\sum_{\sigma \in S} S(\sigma) \cdot (|T_V| - x_\sigma)}{(|T_V| - 1) \cdot \sum_{\sigma \in S} S(\sigma)}$$

Source: [3]

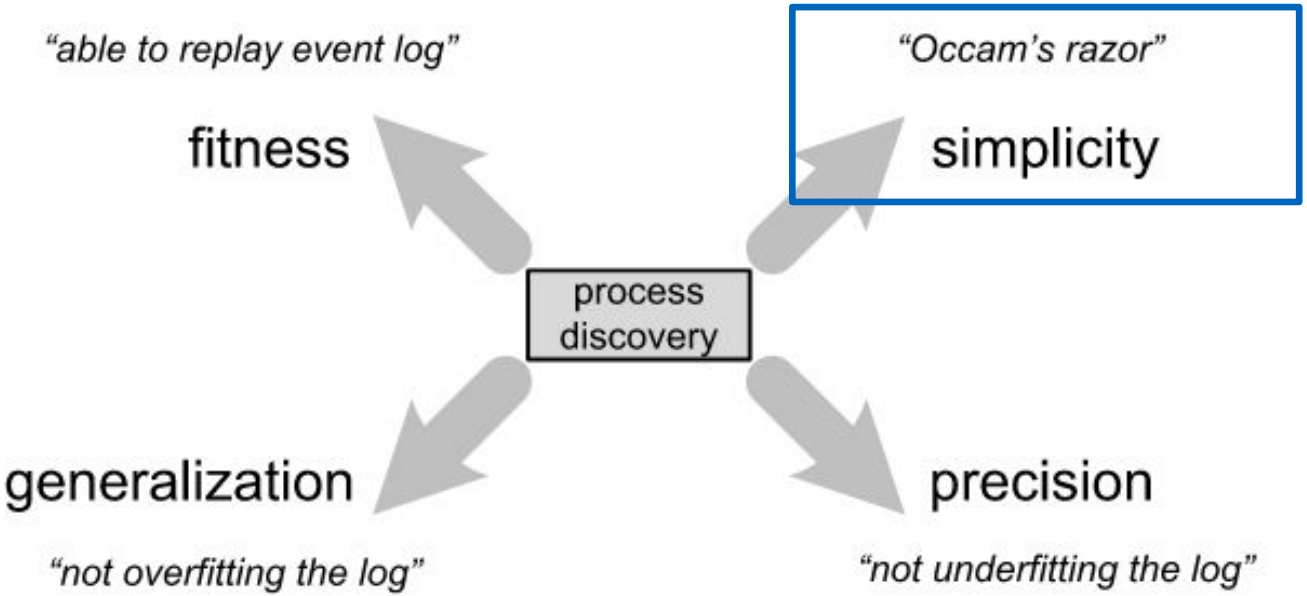
Precision and Generalization

How to measure precision / generalization?

> Check more more metrics on how to measure precision and generalization!

Quality Criteria

Next up: simplicity

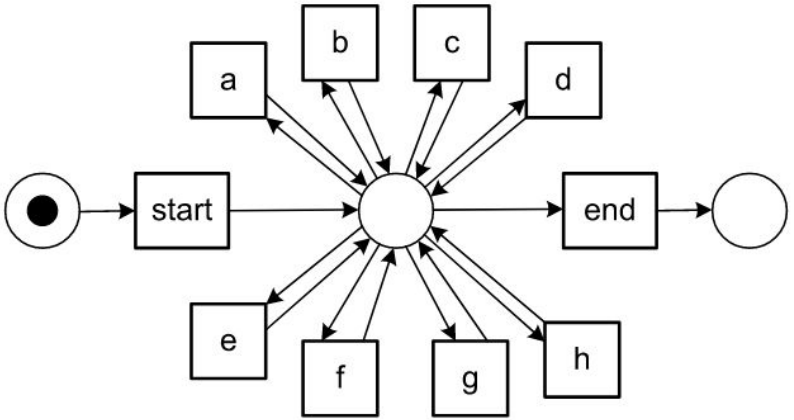


Source: [1]

Simplicity

How understandable is the model to a human

- Simplicity has previously been called structure: *Is the model small and clean enough to read?*
- **Occam's Razor:** The simplest model that can explain the behavior seen in the event log, is the best model.



Flower Petri net: Very simple by many metrics but also readable by human?

Source: [1]

Simplicity

How to measure simplicity?

Metric 6 (Simple Structural Appropriateness) *Given a labeled Petri net* (P, T, F, L, l)*, the simple structural appropriateness metric* a_S *is defined as follows:*

$$a_S = \frac{|\{l(t) \mid t \in T\}| + 2}{|T \cup P|}$$

l(t) := label of transition

Source: [3]

Simplicity

How to measure simplicity?

Metric 7 (Advanced Structural Appropriateness) *Given a labeled Petri net* (P, T, F, L, l)*, let* $T_{DA} \subseteq T$ *be the set of alternative duplicate tasks, and* $T_{IR} \subseteq T$ *the set of redundant invisible tasks. The advanced structural appropriateness metric* a'_S *is defined as follows:*

$$a'_S = \frac{|T| - (|T_{DA}| + |T_{IR}|)}{|T|}$$

Source: [3]

Simplicity

How to measure simplicity?

> Different metrics see [2]



Computer Standards & Interfaces 51 (2017) 104–117

Contents lists available at ScienceDirect

Computer Standards & Interfaces

journal homepage: www.elsevier.com/locate/csi

Complexity metrics for process models – A systematic literature review

Gregor Polančič*, Blaž Cegnar

University of Maribor, Institute of informatics, Smetanova 17, 2000 Maribor, Slovenia

ARTICLE INFO

Keywords:
Process model
Process diagram
Complexity measurement
Metrics
Systematic literature review

ABSTRACT

Context: One of the focal purposes of using 'visual' process models (i.e. process diagrams) is to ensure easier, universally understood and unambiguous diagrammatic communication. Thus the models should be easy to comprehend and maintain, which is directly related to their complexity. In order to systematically address process models complexity, it has to be measured.
Objective: The goal of our work was to provide a better overview and understanding in the field of process models complexity and to provide an overview of the corresponding metrics.
Method: A systematic literature review (SLR) was conducted, being the most suitable method for achieving aforementioned goals. In addition, to answer the stated research questions, different techniques for qualitative and quantitative data analysis and synthesis were used.
Results: We identified 43 relevant articles which were systematically analyzed according to a pre-defined process and data acquisition form. Out of these articles we collected 66 process models complexity metrics.
Conclusion: Modelers can use the 'catalogue' of process complexity metrics to establish and ensure good quality of diagrams, whereas researches can relate to or extend the 'catalogue' by providing new metrics or new insights to existing ones.

1. Introduction

Each organization delivers products or services to its stakeholders by performing different types of interrelated business activities. This corresponds to the definition of a business process, which can be defined as *"a collection of related, structured activities or tasks that produce a specific service or product for a particular customer."*[1] In a "process-aware" company, business processes are commonly seen as the most valuable organizational assets, since they create value for customers by connecting other types of organizational assets into a controlled flow of activities. Additionally, processes are important because they represent intangible organizational assets (i.e. capabilities), which define the productive capacity of an organization (according to ITIL [2] capabilities coordinate, control and deploy resources). This also means that processes cannot be easily acquired – they have to

A process diagram (a diagram is commonly defined as *"a representation in which information is indexed by 2D location"*[4]) is a visual process model and as such it leverages the benefits of visual languages (i.e. visual notations). Visual notations form an integral part of the language of software engineering (SE) and tend to be effective because they tap into the capabilities of the powerful and highly parallel human visual system [4]. According to this, the main purpose of business process diagrams is to provide means for easier and more effective communication, where the effectiveness of 'diagrammatic communication' is measured by the match between the intended message and received message [4].

The importance of 'diagrammatic communication' in business process domain is also evident from the primary goal of BPMN, which states that *"a notation should be readily understandable by all business users."*[5]. Thus it has to be ensured that process diagrams

Quality Criteria: Trade-Off

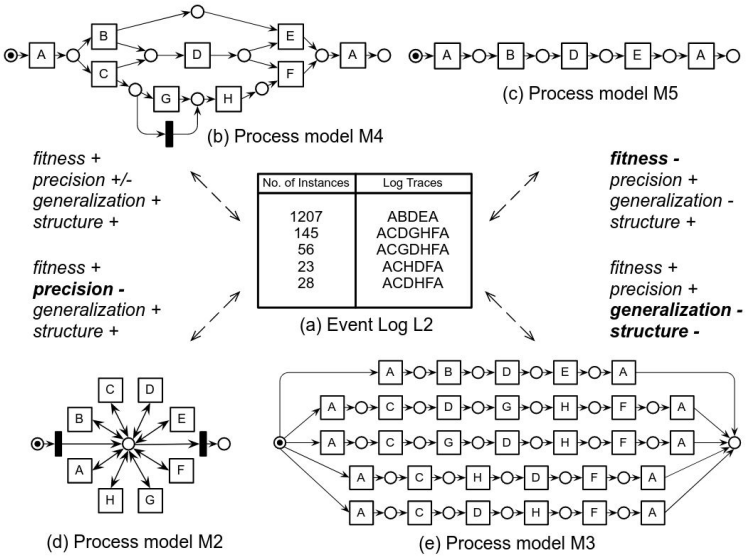
All four metrics must be optimized together, with trade-offs between them

Fitness vs. precision	Increasing fitness means accepting more behaviors which lowers precision
Fitness vs. generalization	Perfectly fitting every trace (including outliers) overfits and hurts generalization.
Precision vs. generalization	Underfitting vs. overfitting
Simplicity vs. fitness	The simplest model has fitness of 1.0 (flower).

> *What about: simplicity vs. generalization?*

Quality Criteria: Examples

Example models for the four criteria



Source: [3]

Tooling: PM4Py

Where to find conformance checking in PM4Py

General	https://pm4py-source.readthedocs.io/en/stable/pm4py.algo.conformance.html#module-pm4py.algo.conformance
Token replay	https://pm4py-source.readthedocs.io/en/latest/pm4py.algo.conformance.tokenreplay.html
Alignments	https://pm4py-source.readthedocs.io/en/stable/pm4py.algo.conformance.alignments.petri_net.html
New	https://processintelligence.solutions/pm4py/api/api/pm4py.algo.conformance.html (depends on your pm4py version)

Tooling: PM4Py

Easy-to-use alternative for conformance checking

pm4py.conformance… https://pm4py-source.readthedocs.io/en/latest/_modules/pm4py/conformance.html

```
def conformance_diagnostics_token_based_replay(log: EventLog, p
                                               final_marking: M
    """
    Apply token-based replay for conformance checking analysis.
    The methods return the full token-based-replay diagnostics.

    Parameters
    --------------
    log
        Event log
    petri_net
        Petri net
    initial_marking
        Initial marking
    final_marking
        Final marking

    Returns
    --------------
    replay_results
        A list of replay results for each trace of the log (in
```

```
def conformance_diagnostics_alignments(log: EventLog, *args, multi_processi
    """
    Apply the alignments algorithm between a log and a process model.
    The methods return the full alignment diagnostics.

    Parameters
    --------------
    log
        Event log
    args
        Specification of the process model
    multi_processing
        Boolean value that enables the multiprocessing (default: False)

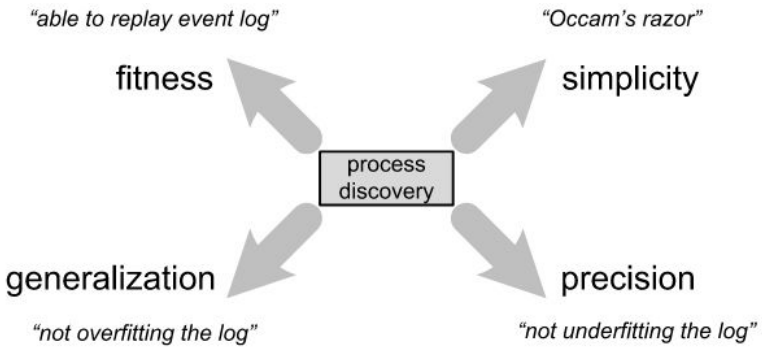
    Returns
    --------------
    aligned_traces
        A list of alignments for each trace of the log (in the same order a
    """
```

Individual Assignment I

Model quality requirement and trade-offs experiment

> Your process model should represent at least 80% of all cases and remain simple while ensuring generalization and precision.

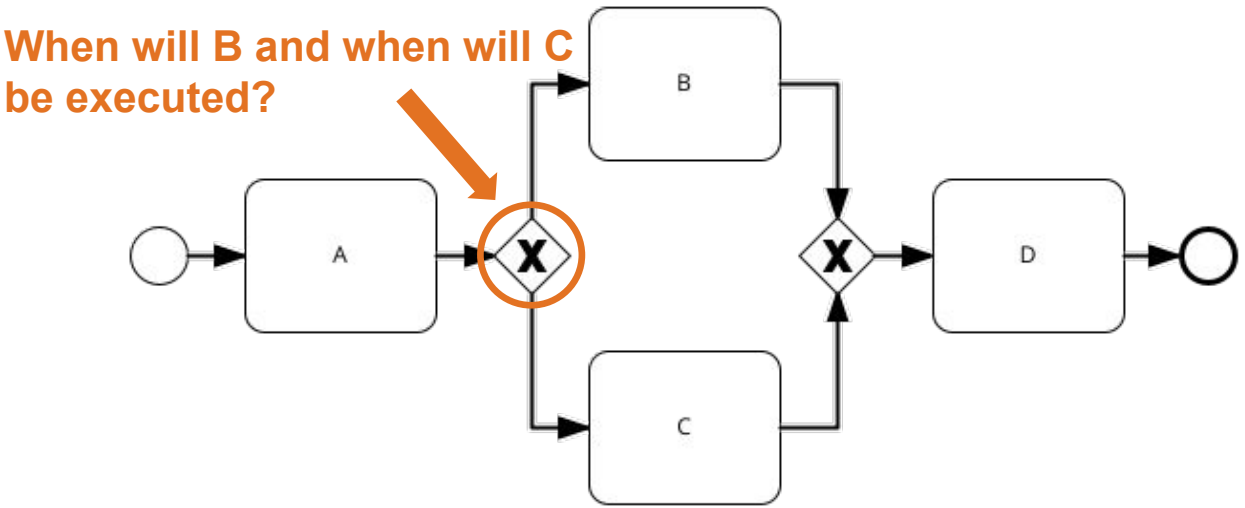
Trade-offs experiment: argue and explain why you optimized for fitness, precision, generalization, and simplicity in the simulation project. Consider both an academic and a consulting/automation perspective, and discuss which objectives are most relevant in each context.



Decision Mining

Decision Mining

Which branch does a case take at a decision point?



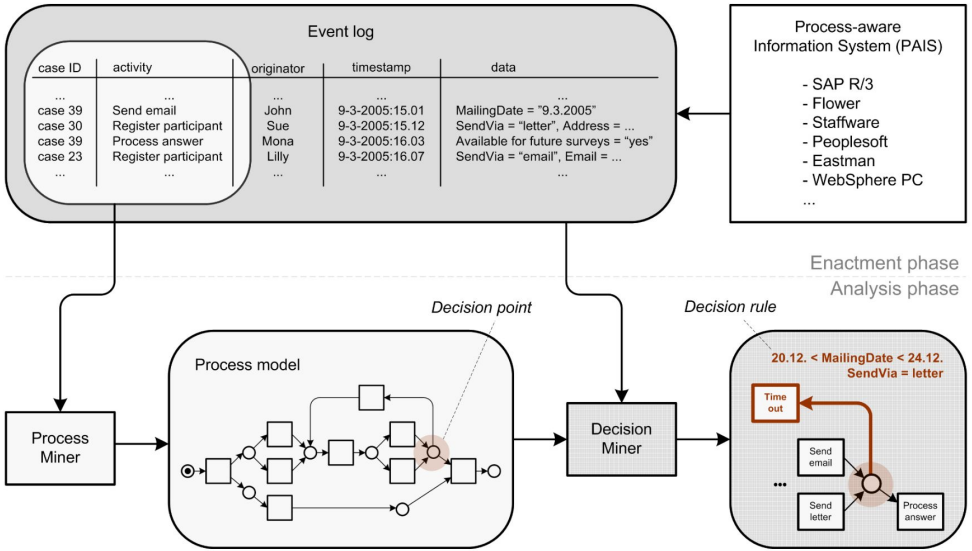
Decision Mining

Decision rules at XOR gateways

Goal	Discover decision rules at XOR gateways that determine, based on case and event attribute data, which activity is executed next at runtime.
Prerequisites	Event log, (~to-be process model).
Required techniques	Process discovery, conformance checking, machine learning based classification models.

Decision Mining

Token-based replay decision mining in business processes



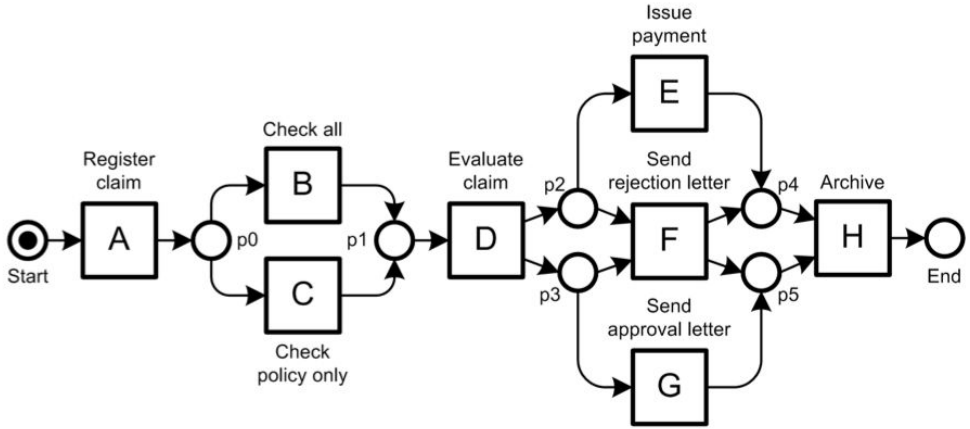
Source: [7]

Decision Mining

1. process discovery: return a Petri net

Case ID	Log trace
1	ACDGEH
2	ABDFH
3	ABDEGH
4	ACDGEH
5	ACDFH
6	ACDFH

(a) Event log



(b) Mined process model

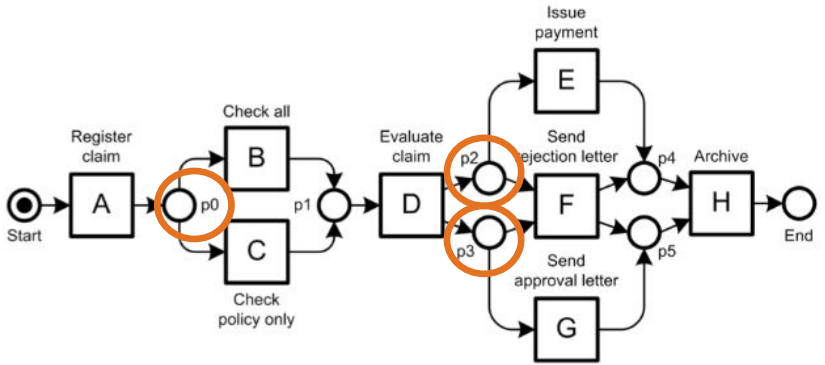
Source: [7]

Decision Mining

2. conformance checking

- Replay traces and find decision points. A decision point is a place in the process with n>=2 outgoing arcs.
- **Why:** Only one transition can consume one token in the place at the same time, therefore a decision need to be taken during runtime.

Decision points:

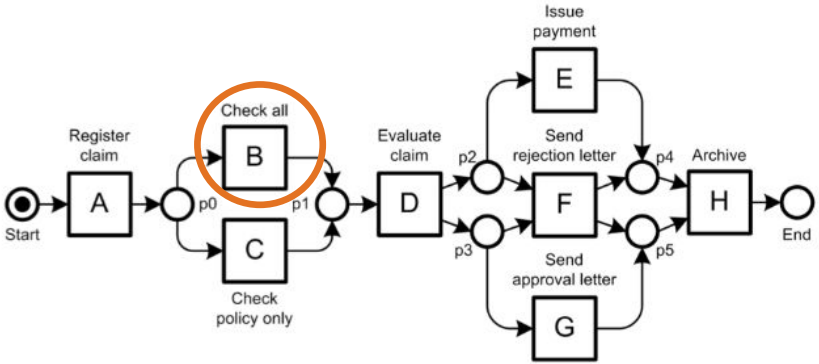


Source: [7]

Decision Mining

3. get training data for each decision point

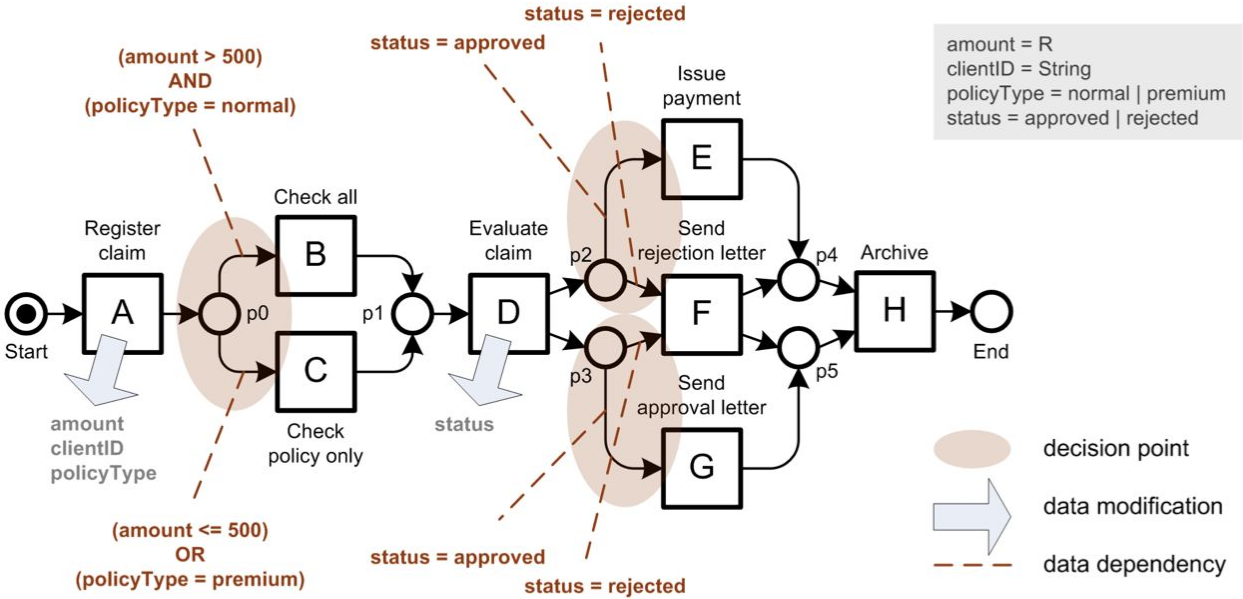
- Approach of [7] uses token-based replay.
- Each trace was replayed: The firing transition after the decision points (e.g., B after p0) is stored as one data value with y: B, and X: all data values (required) up to B (without B).



Source: [7]

Decision Mining

Lastly, get decision rules (guards) from learned decision trees



Source: [7]

Decision Mining: Alignments

Alignment-based decision mining of business processes

Input	Traces and list of moves per trace (alignment)
Process	Create decision (training) data for each decision point. Train decision model
Output	Guards of rules per decision point.

Source: [8]

Algorithm 1: GENERATETRANSITIONGUARDS

Data: $\mathbf{N} = (P, T, F)$ – A Petri net without data, $\mathcal{A}$ – A multi-set of optimal control-flow alignments of N and an event log

Result: A Guard Function $G : T \rightarrow \mathcal{G}$

Let I be a function whose domain is the set of places p s.t. $\mid p^\bullet \mid > 1$ and $\forall p \in P$ s.t. $\mid p^\bullet \mid > 1$. $I(p) = \emptyset$.
foreach alignment $\langle (s_L^1, s_M^1), \ldots, (s_L^n, s_M^n) \rangle \in \mathcal{A}$ **do**
  Set function A such that $\mathsf{dom}(A) = \emptyset$
  for $i \leftarrow 1$ **to** n **do**
    if $s_M^i \neq \gg$ **then**
      foreach $p \in {}^\bullet s_M^i$ *s.t.* $\mid p^\bullet \mid > 1$ **do**
        $I(p) \leftarrow I(p) \cup (A, s_M^i)$
      end
    end
    if $s_L^i \neq \gg \wedge s_M^i \neq \gg$ **then**
      Let $s_L^i = (a_L^i, \phi_L^i)$
      foreach *variable* $v \in \mathit{dom}(\phi_L^i)$ **do**
        $A(v) \leftarrow \phi_L^i(v)$
      end
    end
  end
end
foreach *place* $p \in P$ *s.t.* $\mid p^\bullet \mid > 1$ **do**
  $\psi_p \leftarrow \mathsf{buildFunctionEstimator}(I(p))$
end
foreach *transition* $t \in T$ **do**
  $G(t) \leftarrow \text{true}$
  foreach *place* $p \in {}^\bullet t$ *s.t.* $\mid p^\bullet \mid > 1$ **do**
    $G(t) \leftarrow G(t) \wedge \psi_p(t)$
  end
end
return G

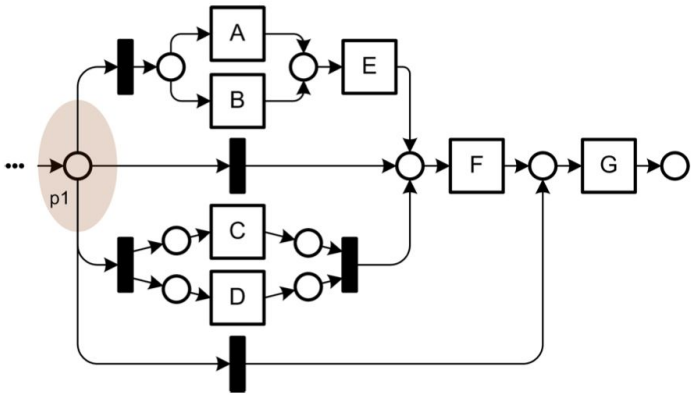
Source: [8]

Decision Mining: Alignments

What alignment-based decision mining solves

Problems Solved

- Can use deviating traces for training.
- Problem of token replay [7]: (chains) of silent transitions after a decision point.
- Find the next sync move after the decision point after the decision point and set as y.



Source: [8]

Decision Mining: Train Data

Choices for decision (train) data

Train Data

- All case attributes?
- All event attributes?
- New long tail attributes?: e.g., last k resources used, aggregated time features (e.g., mean/total waiting or service times), frequency-based features, and recent historical patterns within the case.

Decision Mining: Model

Choices for the classification model

Models

- Which classification model do you want to use (e.g., classic decision trees that facilitate rule extraction, as in [7,8])?
- Or a neural networks with XAI-based rule extraction as in [9]?
- Or make your own decision: train a more advanced classifier and use a surrogate model for rule extraction?

Decision Mining: Output

Colored Petri net: an extension of N = (P,T,A)

CPN=(P,T,A,Σ,V,C,G,E,I)

Σ	Finite set of non-empty color : data: amount, etc.
C	Color function: each place gets color set: Tokens on place p must carry values of type C(p)
G	Guard function: Each transition gets boolean expression
E	Arc expression function, assigning each arc an expression whose type is a multiset over C(p), where p is the place attached to the arc.
I	Initialization function, giving the initial marking of each place as a closed expression of type multiset over C(p).

Decision Mining: Output

Colored Petri net: markings, enabled transitions and firing

Marking	A marking assigns to each place p a multiset of tokens of color C(p).
Enabled transition	A transition t is enabled under a binding bb b (an assignment of values to the variables appearing around t iff (i) the guard G(t) evaluates to true under b, and (ii) every input place contains the tokens demanded by the corresponding arc expression evaluated under bb b.
Firing	Firing (t,b) removes those input tokens and produces, on each output place, the tokens given by the output arc expression evaluated under b.

Decision Mining: SOTA

Newest paper on decision mining (cf. [9])

Neural Networks	XAI
It uses neural networks instead of DT	It uses XAI instead of decision rules.

> Probably this deliver worse explainable rules while maintaining better accuracy.
> *Argue the advantages and disadvantages.*

Source: [9]

Decision Mining: Output

Expected output for each decision point

1 Train

For each decision point in the petri net train an accurate decision model (classifier) that can predict the next activity in the branch.

2 Extract

For each decision point extract decision rules.

3 Translate

Translate decision points to XORs in BPMN and add the rules to the BPMN

Individual Assignment I

Role of decision mining for Bachelor's and Master's students

Bachelor's	Master's
Decision Mining **counts as Advanced Analysis** in Individual Assignment I.	Decision Mining is **mandatory** in Individual Assignment I.

Outline for the Next Meeting…

Preview of the next session

Next topic: Business Process Simulation (19 November 2026)

Literature

References [1] to [4]

[1] van Der Aalst, W., Process Mining: Data science in action. Springer Berlin Heidelberg (2016).

[2] Polančič, G., & Cegnar, B. (2017). Complexity metrics for process models–A systematic literature review. Computer Standards & Interfaces, 51, 104-117.

[3] Rozinat, A. (2010). Process mining: conformance and extension.

[4] Carmona, J., van Dongen, B. F., Solti, A., & Weidlich, M. Conformance checking—relating processes and models. 2018.

Literature

References [5] to [7]

[5] Rozinat, A., & Van der Aalst, W. M. (2008). Conformance checking of processes based on monitoring real behavior. Information Systems, 33(1), 64-95.

[6] Van der Aalst, W., Adriansyah, A., & Van Dongen, B. (2012). Replaying history on process models for conformance checking and performance analysis. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 2(2), 182-192.

[7] Rozinat, A., & Aalst, van der, W. M. P. (2006). Decision mining in business processes. (BETA publicatie : working papers; Vol. 164). Technische Universiteit Eindhoven.

Literature

References [8] to [9]

[8] De Leoni, M., & Van Der Aalst, W. M. (2013). Data-aware process mining: discovering decisions in processes using alignments. In Proceedings of the 28th annual ACM symposium on applied computing (pp. 1454-1461).

[9] Park, G., Küsters, A., Tews, M., Pitsch, C., Schneider, J., & van der Aalst, W. M. (2022). Exlainable predictive decision mining for operational support. In International Conference on Service-Oriented Computing (pp. 66-79). Cham: Springer Nature Switzerland.